

Ex. No. 2	BASIC SQL
Date of Exercise	24/07/2026

Aim

To execute the given basic SQL queries.

Description

DML (Data Manipulation Language) statements manipulate the data held within tables: INSERT adds rows, UPDATE changes existing rows, DELETE removes rows, and SELECT retrieves rows, optionally filtered with a WHERE clause (comparison, AND/OR, IS NULL, IN, LIKE, BETWEEN), grouped with GROUP BY, and ordered with ORDER BY.

1. Insert all the fields given in the ticket reservation schema.**Sample Code:**

```
INSERT INTO URK24CS1021_users (id, name, email, password, phone) VALUES
  (101, 'Alice Johnson', 'alice@mail.com', 'pass1', '9000000001'),
  (102, 'Bob Smith', 'bob@mail.com', 'pass2', '9000000002'),
  (103, 'Carol Lee', 'carol@mail.com', 'pass3', '9000000003'),
  (104, 'David Kim', 'david@mail.com', 'pass4', '9000000004'),
  (105, 'Emma Watson', 'emma@mail.com', 'pass5', '9000000005');
INSERT INTO URK24CS1021_location (venueid, name, address, city, state, country)
VALUES
  (1, 'Madison Square Garden', '4 Pennsylvania Plaza', 'New York', 'NY',
  'USA'),
  (2, 'Wembley Stadium', 'Wembley', 'London', 'England', 'UK'),
  (3, 'Marina Bay Sands', '10 Bayfront Ave', 'Singapore', 'Singapore',
  'Singapore');
INSERT INTO URK24CS1021_event (eventid, name, eventdate, eventtime, venueid,
description) VALUES
  (1, 'Rock Fest', '2023-08-15', '2023-08-15 19:00:00', 1, 'Outdoor rock
concert'),
  (2, 'Tech Conference', '2023-07-20', '2023-07-20 09:00:00', 2, 'Annual tech
meetup'),
  (3, 'Jazz Night', '2023-09-05', '2023-09-05 20:00:00', 3, 'Live jazz
performance'),
  (4, 'Startup Summit', '2023-06-10', '2023-06-10 10:00:00', 1, 'Startup pitch
day');
INSERT INTO URK24CS1021_ticket (ticketid, eventid, userid, price, status,
barcode) VALUES
  (1, 1, 101, 150.00, 'confirmed', 'BC1001'),
  (2, 1, 102, 150.00, 'confirmed', 'BC1002'),
  (3, 2, 103, 200.00, 'cancelled', 'BC1003'),
  (4, 3, 104, 120.00, 'confirmed', 'BC1004'),
  (5, 3, 105, 120.00, 'confirmed', 'BC1005'),
  (6, 4, 101, 90.00, 'cancelled', 'BC1006'),
  (7, 2, 102, 200.00, 'confirmed', 'BC1007');
```

Sample Output:

```
psql — labdb

labdb=# INSERT INTO URK24CS1021_users (id, name, email, password, phone) VALUES
labdb(#      (101, 'Alice Johnson', 'alice@mail.com', 'pass1', '9000000001'),
labdb(#      (102, 'Bob Smith', 'bob@mail.com', 'pass2', '9000000002'),
labdb(#      (103, 'Carol Lee', 'carol@mail.com', 'pass3', '9000000003'),
labdb(#      (104, 'David Kim', 'david@mail.com', 'pass4', '9000000004'),
labdb(#      (105, 'Emma Watson', 'emma@mail.com', 'pass5', '9000000005');
INSERT 0 5

labdb=# INSERT INTO URK24CS1021_location (venueid, name, address, city, state, country) VALUES
labdb(#      (1, 'Madison Square Garden', '4 Pennsylvania Plaza', 'New York', 'NY', 'USA'),
labdb(#      (2, 'Wembley Stadium', 'Wembley', 'London', 'England', 'UK'),
labdb(#      (3, 'Marina Bay Sands', '10 Bayfront Ave', 'Singapore', 'Singapore', 'Singapore');
INSERT 0 3

labdb=# INSERT INTO URK24CS1021_event (eventid, name, eventdate, eventtime, venueid, description) VALUES
labdb(#      (1, 'Rock Fest', '2023-08-15', '2023-08-15 19:00:00', 1, 'Outdoor rock concert'),
labdb(#      (2, 'Tech Conference', '2023-07-20', '2023-07-20 09:00:00', 2, 'Annual tech meetup'),
labdb(#      (3, 'Jazz Night', '2023-09-05', '2023-09-05 20:00:00', 3, 'Live jazz performance'),
labdb(#      (4, 'Startup Summit', '2023-06-10', '2023-06-10 10:00:00', 1, 'Startup pitch day');
INSERT 0 4

labdb=# INSERT INTO URK24CS1021_ticket (ticketid, eventid, userid, price, status, barcode) VALUES
labdb(#      (1, 1, 101, 150.00, 'confirmed', 'BC1001'),
labdb(#      (2, 1, 102, 150.00, 'confirmed', 'BC1002'),
labdb(#      (3, 2, 103, 200.00, 'cancelled', 'BC1003'),
labdb(#      (4, 3, 104, 120.00, 'confirmed', 'BC1004'),
labdb(#      (5, 3, 105, 120.00, 'confirmed', 'BC1005'),
labdb(#      (6, 4, 101, 90.00, 'cancelled', 'BC1006'),
labdb(#      (7, 2, 102, 200.00, 'confirmed', 'BC1007');
INSERT 0 7

labdb=#
```

2. Display all the fields in User, Events, Venue and Ticket tables.**Sample Code:**

```
SELECT * FROM URK24CS1021_users;
SELECT * FROM URK24CS1021_event;
SELECT * FROM URK24CS1021_location;
SELECT * FROM URK24CS1021_ticket;
```

Sample Output:

```
psql — labdb

labdb=# SELECT * FROM URK24CS1021_users;
 id |   name   |   email   | password |   phone
-----+-----+-----+-----+-----
 101 | Alice Johnson | alice@mail.com | pass1   | 9000000001
 102 | Bob Smith   | bob@mail.com   | pass2   | 9000000002
 103 | Carol Lee   | carol@mail.com | pass3   | 9000000003
 104 | David Kim   | david@mail.com | pass4   | 9000000004
 105 | Emma Watson | emma@mail.com  | pass5   | 9000000005
(5 rows)

labdb=# SELECT * FROM URK24CS1021_event;
 eventid |   name   | eventdate | eventtime | venueid | description
-----+-----+-----+-----+-----+-----
       1 | Rock Fest | 2023-08-15 | 2023-08-15 19:00:00 |      1 | Outdoor rock concert
       2 | Tech Conference | 2023-07-20 | 2023-07-20 09:00:00 |      2 | Annual tech meetup
       3 | Jazz Night | 2023-09-05 | 2023-09-05 20:00:00 |      3 | Live jazz performance
       4 | Startup Summit | 2023-06-10 | 2023-06-10 10:00:00 |      1 | Startup pitch day
(4 rows)

labdb=# SELECT * FROM URK24CS1021_location;
 venueid |   name   | address | city | state | country
-----+-----+-----+-----+-----+-----
       1 | Madison Square Garden | 4 Pennsylvania Plaza | New York | NY | USA
       2 | Wembley Stadium | Wembley | London | England | UK
       3 | Marina Bay Sands | 10 Bayfront Ave | Singapore | Singapore | Singapore
(3 rows)

labdb=# SELECT * FROM URK24CS1021_ticket;
 ticketid | eventid | userid | price | status | barcode
-----+-----+-----+-----+-----+-----
        1 |      1 |    101 | 150.00 | confirmed | BC1001
        2 |      1 |    102 | 150.00 | confirmed | BC1002
        3 |      2 |    103 | 200.00 | cancelled | BC1003
        4 |      3 |    104 | 120.00 | confirmed | BC1004
        5 |      3 |    105 | 120.00 | confirmed | BC1005
        6 |      4 |    101 |  90.00 | cancelled | BC1006
        7 |      2 |    102 | 200.00 | confirmed | BC1007
(7 rows)

labdb=#
```

3. Retrieve all venues ordered by city and then by country.

Sample Code:

```
SELECT * FROM URK24CS1021_location ORDER BY city, country;
```

Sample Output:

```
psql — labdb

labdb=# SELECT * FROM URK24CS1021_location ORDER BY city, country;
 venueid |   name   | address | city | state | country
-----+-----+-----+-----+-----+-----
       2 | Wembley Stadium | Wembley | London | England | UK
       1 | Madison Square Garden | 4 Pennsylvania Plaza | New York | NY | USA
       3 | Marina Bay Sands | 10 Bayfront Ave | Singapore | Singapore | Singapore
(3 rows)

labdb=#
```

4. Retrieve total number of users.

Sample Code:

```
SELECT COUNT(*) AS total_users FROM URK24CS1021_users;
```

Sample Output:

```
psql — labdb

labdb=# SELECT COUNT(*) AS total_users FROM URK24CS1021_users;
 total_users 
-----
           5
(1 row)

labdb=#
```

5. Retrieve all events ordered by date and time in descending order.**Sample Code:**

```
SELECT * FROM URK24CS1021_event ORDER BY eventdate DESC, eventtime DESC;
```

Sample Output:

```
psql — labdb

labdb=# SELECT * FROM URK24CS1021_event ORDER BY eventdate DESC, eventtime DESC;
 eventid |   name   | eventdate | eventtime | venueid | description 
-----+-----+-----+-----+-----+-----
        3 | Jazz Night | 2023-09-05 | 2023-09-05 20:00:00 |        3 | Live jazz performance
        1 | Rock Fest  | 2023-08-15 | 2023-08-15 19:00:00 |        1 | Outdoor rock concert
        2 | Tech Conference | 2023-07-20 | 2023-07-20 09:00:00 |        2 | Annual tech meetup
        4 | Startup Summit | 2023-06-10 | 2023-06-10 10:00:00 |        1 | Startup pitch day
(4 rows)

labdb=#
```

6. Get the average price of tickets for each event.**Sample Code:**

```
SELECT eventid, AVG(price) AS avg_price
FROM URK24CS1021_ticket GROUP BY eventid;
```

Sample Output:

```
psql — labdb

labdb=# SELECT eventid, AVG(price) AS avg_price
labdb(# FROM URK24CS1021_ticket GROUP BY eventid;
 eventid |      avg_price
-----+-----
          3 | 120.0000000000000000
          1 | 150.0000000000000000
          4 |  90.0000000000000000
          2 | 200.0000000000000000
(4 rows)

labdb=#
```

7. Get the total price and count of tickets for each event.

Sample Code:

```
SELECT eventid, SUM(price) AS total_price, COUNT(*) AS ticket_count
FROM URK24CS1021_ticket GROUP BY eventid;
```

Sample Output:

```
psql — labdb

labdb=# SELECT eventid, SUM(price) AS total_price, COUNT(*) AS ticket_count
labdb(# FROM URK24CS1021_ticket GROUP BY eventid;
 eventid | total_price | ticket_count
-----+-----+-----
          3 |      240.00 |           2
          1 |      300.00 |           2
          4 |       90.00 |           1
          2 |      400.00 |           2
(4 rows)

labdb=#
```

8. Count the number of tickets for each event.

Sample Code:

```
SELECT eventid, COUNT(*) AS ticket_count
FROM URK24CS1021_ticket GROUP BY eventid;
```

Sample Output:

```
psql — labdb

labdb=# SELECT eventid, COUNT(*) AS ticket_count
labdb(# FROM URK24CS1021_ticket GROUP BY eventid;
 eventid | ticket_count
-----+-----
          3 |          2
          1 |          2
          4 |          1
          2 |          2
(4 rows)

labdb=#
```

9. Retrieve a distinct number of users.**Sample Code:**

```
SELECT COUNT(DISTINCT id) AS distinct_users FROM URK24CS1021_users;
```

Sample Output:

```
psql — labdb

labdb=# SELECT COUNT(DISTINCT id) AS distinct_users FROM URK24CS1021_users;
 distinct_users
-----
              5
(1 row)

labdb=#
```

10. Retrieve the events conducted after 30/07/2023.**Sample Code:**

```
SELECT * FROM URK24CS1021_event WHERE eventdate > '2023-07-30';
```

Sample Output:

```
psql — labdb

labdb=# SELECT * FROM URK24CS1021_event WHERE eventdate > '2023-07-30';
 eventid |   name   | eventdate | eventtime | venueid | description
-----+-----+-----+-----+-----+-----
       1 | Rock Fest | 2023-08-15 | 2023-08-15 19:00:00 |       1 | Outdoor rock concert
       3 | Jazz Night | 2023-09-05 | 2023-09-05 20:00:00 |       3 | Live jazz performance
(2 rows)

labdb=#
```

11. Calculate the sum of prices for all confirmed tickets.

Sample Code:

```
SELECT SUM(price) AS confirmed_total
FROM URK24CS1021_ticket WHERE status = 'confirmed';
```

Sample Output:

```
psql — labdb

labdb=# SELECT SUM(price) AS confirmed_total
labdb=# FROM URK24CS1021_ticket WHERE status = 'confirmed';
 confirmed_total
-----
          740.00
(1 row)

labdb=#
```

12. Concatenate the user's name and email for display.

Sample Code:

```
SELECT name || ' <' || email || '>' AS display FROM URK24CS1021_users;
```

Sample Output:

```
psql — labdb

labdb=# SELECT name || ' <' || email || '>' AS display FROM URK24CS1021_users;
 display
-----
Alice Johnson <alice@mail.com>
Bob Smith <bob@mail.com>
Carol Lee <carol@mail.com>
David Kim <david@mail.com>
Emma Watson <emma@mail.com>
(5 rows)

labdb=#
```

13. Retrieve the venue from the city New York and country USA.**Sample Code:**

```
SELECT * FROM URK24CS1021_location
WHERE city = 'New York' AND country = 'USA';
```

Sample Output:

```
psql — labdb

labdb=# SELECT * FROM URK24CS1021_location
labdb(# WHERE city = 'New York' AND country = 'USA';
 venueid |          name          |          address          | city  | state | country
-----+-----+-----+-----+-----+-----
        1 | Madison Square Garden | 4 Pennsylvania Plaza | New York | NY    | USA
(1 row)

labdb=#
```

14. Retrieve all cities ordered by Country in ascending order.**Sample Code:**

```
SELECT city, country FROM URK24CS1021_location ORDER BY country ASC;
```

Sample Output:

```
psql — labdb

labdb=# SELECT city, country FROM URK24CS1021_location ORDER BY country ASC;
  city  | country
-----+-----
Singapore | Singapore
London  | UK
New York | USA
(3 rows)

labdb=#
```

15. Delete the cancelled ticket in the ticket table.**Sample Code:**

```
DELETE FROM URK24CS1021_ticket WHERE status = 'cancelled';
```

Sample Output:

```
psql — labdb

labdb=# DELETE FROM URK24CS1021_ticket WHERE status = 'cancelled';
DELETE 2

labdb=#
```

RESULT

The SQL queries were executed and the desired output was obtained.